

**Национальный исследовательский ядерный
университет «МИФИ»**

Проектная практика

**Тема работы:
«Система распознавания типа комнаты
по изображению (PJ)»**

Выполнили: студенты группы М25-525

**Давид Сухашвили,
Алексей Тепляков,
Лина Цаплева,
Дарья Вдовина,
Михаил Калмыков,
Глеб Медведев**

Содержание

1. Описание задания.....	3
1.1. Описание задачи.....	3
1.2. Оценивание.....	3
2. Обоснование проекта и возможные сложности.....	4
3. Общая архитектура проекта.....	5
4. Подготовка данных.....	6
4.1. Входные файлы.....	6
4.2. Логика make_manifest.py.....	6
5. Weak labels и дополнительные данные.....	7
6. Training pipeline: src/training/train_image.py.....	8
6.1. Dataset/DataLoader.....	9
6.2. Аугментации.....	9
6.3. Loss, class weights и sampler.....	9
6.4. Early stopping и checkpoint.....	9
7. Модели и эксперименты.....	10
8. Финальная модель: cv05_rtx5070_xlarge_512_focal_es.....	11
9. Полученные результаты и метрики.....	12
10. MLflow и управление экспериментами.....	13
11. Артефакты обучения и формат OOF.....	14
12. Inference pipeline.....	15
13. Demo: app.py на Gradio.....	15
14. Основные команды запуска.....	16
15. Итоговые выводы.....	17

1. Описание задания

1.1. Описание задачи

Необходимо решить задачу классификации изображений: обучить модель, которая по изображению определяет тип комнаты. Требуется предсказать один из 19 классов:

- 0 — кухня / столовая
- 1 — кухня-гостиная
- 2 — универсальная комната
- 3 — гостиная
- 4 — спальня
- 5 — кабинет
- 6 — детская
- 7 — ванная комната
- 8 — туалет
- 9 — совмещённый санузел
- 10 — коридор / прихожая
- 11 — гардеробная / кладовая / постирочная
- 12 — балкон / лоджия
- 13 — вид из окна / с балкона
- 14 — дом снаружи / двор
- 15 — подъезд / лестничная площадка
- 16 — другое
- 17 — предметы интерьера / бытовая техника
- 18 — комната без мебели

1.2. Оценивание

Метрика — Macro F1-score: считается F1 для каждого класса, затем берётся невзвешенное среднее.

2. Обоснование проекта и возможные сложности

Цель проекта - построить воспроизводимый ML/CV pipeline для классификации изображений помещений. Такая задача актуальна для объявлений недвижимости: по фотографии можно автоматически определить тип комнаты и использовать это для поиска, модерации, аналитики, улучшения карточек объявлений и контроля качества данных.

Главная сложность задачи - визуальная близость классов. Например, спальня, гостиная и универсальная комната могут выглядеть похожими; гардеробная, кладовая и постирочная также часто имеют пересекающиеся признаки. Поэтому в проекте использовались сильные предобученные backbone-модели, высокое разрешение изображения, балансировка классов, Focal Loss и строгий контроль метрик.

Таблица 1. Проблемы и решения

Проблема	Решение в проекте
Несбалансированные классы	macro F1 как главная метрика, balanced sampler, weighted CE, Focal Loss
Визуально похожие классы	ConvNeXt, увеличение входа до 512x512, аугментации safe_v1
Риск битых/отсутствующих изображений	make_manifest.py, проверка PIL, sha256, отчеты missing/corrupted
Невоспроизводимость экспериментов	YAML-конфиги, split_version, dataset_version, seed, MLflow
Сложность сравнения моделей	OOF/parquet-артефакты, единый формат logits/probabilities
Нужна демонстрация результата	Gradio app.py и RoomPredictor

3. Общая архитектура проекта

Проект построен как классический end-to-end CV pipeline: данные проходят проверку, затем формируются split-ы, запускаются эксперименты обучения, результаты логируются в MLflow, лучшая модель экспортируется для inference и демонстрируется через Gradio/API.

Данные

- > make_manifest.py: проверка файлов, hash, размеры, отчеты
- > split generation: train/validation/shadow holdout
- > train_image.py: Dataset/DataLoader, transforms, model, loss, sampler
- > MLflow: параметры, метрики, артефакты
- > OOF/shadow predictions: parquet с logits/probabilities
- > release config + checkpoint
- > RoomPredictor / predict.py / app.py
- > demo UI или API endpoint /predict

Таблица 2: Архитектура проекта

Директория	Назначение
configs/model	YAML-конфиги экспериментов обучения
configs/release	Конфиги релизной модели для inference/demo
data/raw	Исходные CSV и изображения train/val/test
data/processed	Технические артефакты данных: manifest, weak labels
data/splits	Фиксированные split-ы и fold-ы
src/datasets	Подготовка датасета, manifest, weak labels
src/training	Основной training pipeline
src/inference	RoomPredictor и запуск предсказаний
src/experiments	Метрики, таблицы, отчеты
artifacts/checkpoints	Сохраненные веса моделей
artifacts/oof	OOF и shadow predictions
artifacts/logs	MLflow database и артефакты логирования
reports	Markdown-отчеты по данным и экспериментам

4. Подготовка данных

Подготовка данных была вынесена в отдельный модуль `make_manifest.py`. Его задача - до обучения проверить, что все изображения существуют, корректно открываются, имеют ожидаемые имена и могут быть связаны со строками CSV.

4.1. Входные файлы

Таблица 3: Входные данные

Split	CSV-файл	Папка с изображениями
train	data/raw/train_df.csv	data/raw/train_images/train_images
val	data/raw/val_df.csv	data/raw/val_images/val_images
test	data/raw/test_df.csv	data/raw/test_images/test_images

4.2. Логика `make_manifest.py`

CSV-файлы

- > поиск папок `train_images/val_images/test_images`
- > распаковка архивов при необходимости
- > нормализация имени `image_id_ext`: добавление `.jpg` при отсутствии расширения
- > проверка наличия файла
- > попытка скачивания по URL из колонки `image`, если файл отсутствует
- > проверка `PIL Image.open(...).verify()`
- > расчет `sha256`
- > извлечение `width/height`
- > запись `data_manifest.parquet`
- > отчеты `data_integrity.md`, `missing_files.csv`, `corrupted_files.csv`

Функция	Назначение
<code>resolve_base_dir</code>	Находит корневую папку датасета или использует <code>--base-dir</code>
<code>find_split_dir</code>	Учитывает варианты <code>data/raw/train_images</code> и <code>data/raw/train_images/train_images</code>
<code>unpack_split_archive</code>	Распаковывает <code>zip/tar/tgz</code> , если папка с изображениями отсутствует
<code>normalize_image_name</code>	Добавляет <code>.jpg</code> к <code>image_id_ext</code> , если расширение не указано
<code>download_image</code>	Скачивает отсутствующий файл по URL во временный <code>.part</code> и атомарно заменяет файл
<code>sha256_file</code>	Считает <code>sha256</code> для контроля целостности и поиска дублей
<code>check_image</code>	Проверяет файл, присваивает <code>ok/missing/corrupted</code> , сохраняет ошибку
<code>build_report</code>	Формирует markdown-отчет по качеству данных
Колонка <code>manifest</code>	Описание
<code>image_id_ext</code>	Имя изображения с расширением
<code>split</code>	<code>train</code> , <code>val</code> или <code>test</code>
<code>local_path</code>	Локальный путь к файлу
<code>hash_sha256</code>	Хэш файла
<code>width / height</code>	Размер изображения
<code>status</code>	<code>ok</code> , <code>missing</code> или <code>corrupted</code>
<code>error</code>	Текст ошибки при <code>missing/corrupted</code>

```
uv run python src/datasets/make_manifest.py
```

```
uv run python src/datasets/make_manifest.py --base-dir data/raw
```

Важная доработка: в `main()` добавлена проверка, что обработаны все ожидаемые `split`-ы и что суммарное число строк `manifest` совпадает с `train + val + test`. Это защищает от тихого пропуска части данных.

5. Weak labels и дополнительные данные

В проекте рассматривалась поддержка weak supervision: к основному train можно добавлять изображения, размеченные эвристиками. Это полезно для редких классов, но требует пониженного веса и аккуратного контроля источника.

Таблица 4: Weak labels

Источник	Назначение
heuristics_cabinet.csv	Дополнительные примеры класса кабинет/рабочая зона
heuristics_detskaya.csv	Дополнительные примеры класса детская
heuristics_dressing_room.csv	Дополнительные примеры гардеробной/кладовой/постирочной
weak_labels_v1.parquet	Итоговый parquet с weak labels, class_id, source_dataset, weak_weight

В train_image.py weak labels подключаются только при experiment.weak_label_flag=true. В таком случае training frame объединяет обычный train и weak manifest, а веса объектов могут учитываться через source или source_x_ratio sample weighting.

6. Training pipeline: src/training/train_image.py

train_image.py - центральный файл проекта. Он превращает YAML-конфиг и подготовленные split-ы в обученную модель, checkpoint, OOF-предсказания, shadow-предсказания, MLflow-run и markdown-отчет.

```
uv run python src/training/train_image.py --config configs/model/image_baseline_v1.yaml --all-folds
```

```
uv run python src/training/train_image.py --config configs/model/cv05_rtx5070_xlarge_512_focal_es.yaml --fold 0 --debug
```

Таблица 5: Training pipeline

CLI-аргумент	Назначение
--config	Путь к YAML-конфигу эксперимента
--fold	Номер fold для одиночного запуска
--all-folds	Обучить все fold-ы из splits_json
--debug	Быстрый smoke run на малой выборке
--device	auto/cpu/cuda/mps; переопределяет train.device
Блок	Что делает
parse_args	Читает CLI-параметры
set_seed	Фиксирует random/numpy/torch seed
RoomDataset	Возвращает image, label, sample_weight
get_transforms	Строит torchvision или albumentations transforms
create_model	Создает timm-модель с num_classes=20
build_criterion	Создает CE/FocalLoss с учетом label smoothing
train_one_epoch	Один проход обучения
predict	Валидация/OOF/shadow inference
prediction_frame	Собирает parquet с target, pred, logits, prob
run_fold	Полный цикл одного fold
run_image_training	Запускает один или все fold-ы, агрегирует результаты

6.1. Dataset/DataLoader

RoomDataset читает строки DataFrame, выбирает путь изображения из local_path или из images_dir/image_id_ext, открывает PIL.Image, применяет transforms и возвращает тройку image, label, sample_weight. Это позволяет одному pipeline поддерживать обычное обучение, ratio weighting и weak label weighting.

6.2. Аугментации

Таблица 6: Аугментация

Policy	Состав	Назначение
baseline_v1	Resize, RandomResizedCrop, HorizontalFlip, ColorJitter, Normalize	Базовый augmentation pipeline
safe_v1	Crop с ограничениями, Flip, ColorJitter, RandomGrayscale, RandomErasing	Устойчивость к ракурсам, освещению, частичному закрытию
albumentations_v1	Brightness/Contrast, HSV, CLAHE, Affine, ToGray, CoarseDropout	Более богатые аугментации через Albumentations

6.3. Loss, class weights и sampler

Механизм	Варианты	Зачем нужен
Loss	ce, weighted_ce, focal	Стандартное обучение, учет дисбаланса, акцент на сложных примерах
Class weights	raw_inverse, sqrt_inv, effective_num, sqrt_median_effective	Повышение вклада редких классов
Sample weights	none, ratio, source, source_x_ratio	Взвешивание примеров по ratio/weak source
Sampler	shuffle, balanced, class_aware_mixture, repeat_factor	Регулирует частоту попадания классов в batch

6.4. Early stopping и checkpoint

В финальном pipeline мониторится val_macro_f1. Если метрика не улучшается на min_delta в течение patience эпох, обучение останавливается. Лучший checkpoint сохраняет model_state_dict, backbone, num_classes, fold, epoch, best_f1, monitor_metric и config.

```
early_stopping:
  enabled: true
  patience: 10
  min_delta: 0.001
  monitor: val_macro_f1
  mode: max
```

7. Модели и эксперименты

Таблица 7: Модели и эксперименты

Эксперимент / модель	Что проверялось
baseline_v1	Базовый ConvNeXt-Tiny, стандартная CE-логика
cv03_baseline_ce	Контрольный baseline для CV-03
cv03_weighted_ce	CrossEntropy с весами классов
cv03_balanced_sampler	Балансировка sampling редких классов
cv03_ratio_weighting	Взвешивание объектов по ratio
cv03_focal_loss	Focal Loss для сложных/редких примеров
alumentations_v1	Проверка расширенных аугментаций
model2_v1	EfficientNet-B0 как легкая альтернативная модель
cv04_convnext_small_earlystop_gpu	ConvNeXt-Small с early stopping
cv05_rtx5070_xlarge_512_focal_es	Финальная тяжелая модель: ConvNeXt XLarge, 512, focal, balanced
cv05/cv06 CLIP	Эксперименты с CLIP ViT-B/32 и ViT-L/14

8. Финальная модель:

cv05_rtx5070_xlarge_512_focal_es

Финальная конфигурация ориентирована на максимальное качество: крупный backbone ConvNeXt XLarge, вход 512x512, Focal Loss, balanced sampler, CUDA, AMP и gradient checkpointing. Batch size равен 1 из-за высокого потребления видеопамати.

Таблица 8: Финальная модель

Параметр	Значение
Backbone	convnext_xlarge.fb_in22k_ft_in1k_384
Pretrained	true
Количество классов	20
image_size / resize_size	512 / 576
Loss	focal, gamma=2.0
Sampler	balanced
Optimizer	AdamW
lr / weight_decay	0.00002 / 0.05
AMP	true
Gradient checkpointing	true
Early stopping	patience=10, min_delta=0.001, monitor=val_macro_f1
MLflow experiment	room_type_image_heavy
OOF dir	artifacts/oof/cv05_rtx5070_xlarge_512_focal_es

```
data:
  image_size: 512
  resize_size: 576
model:
  backbone: convnext_xlarge.fb_in22k_ft_in1k_384
  pretrained: true
  grad_checkpointing: true
train:
  epochs: 1000
  batch_size: 1
  lr: 0.00002
  weight_decay: 0.05
  amp: true
  device: cuda
experiment:
  loss: focal
  focal_gamma: 2.0
  sampler: balanced
  feature_flags: img_heavy_512
```

9. Полученные результаты и метрики

Основной метрикой проекта выбрана макро F1, потому что она усредняет качество по классам и лучше отражает ситуацию при дисбалансе, чем обычная ассигасу.

Таблица 9: Результаты

Run	train_loss	val_loss	val_macro_f1	val_accuracy	best_val_macro_f1	final_oof_macro_f1	final_shadow_macro_f1	final_shadow_accuracy	ES epoch
Run #1	0.0094	0.5756	0.7563	0.7708	0.7934	0.7934	0.7314	0.7694	21
Run #2	0.0139	0.6115	0.7488	0.7566	0.7694	0.7694	0.7284	0.7694	19
Run #3	0.0084	0.6862	0.7400	0.7427	0.7662	0.7662	0.7466	0.7862	23
Run #4	0.0117	0.6695	0.7407	0.7664	0.7648	0.7648	0.7372	0.7694	20
Run #5	0.0173	0.6508	0.7286	0.7423	0.7595	0.7595	0.7584	0.8024	14

Лучший результат по best_val_macro_f1 и final_oof_macro_f1 показал Run #1: около 0.7934.

Лучший shadow ассигасу показал Run #5: около 0.8024, но его макро F1 был ниже, что может указывать на более сильную ориентацию на частые классы.

Таблица 10: Выводы

Вывод	Интерпретация
ConvNeXt XLarge + 512 дает лучший потенциал качества	Высокое разрешение помогает сохранить мелкие признаки: мебель, сантехнику, кухонные элементы
Focal Loss полезен при дисбалансе	Снижает вклад простых примеров и усиливает сложные
Balanced sampler повышает внимание к редким классам	Особенно важно для макро F1
Early stopping работал эффективно	Фактическая остановка около 14-23 эпох при максимуме 1000
Остаются сложные классы	Универсальная комната, гостиная, спальня, кладовая/гардеробная, неясные изображения

10. MLflow и управление экспериментами

MLflow используется для централизованного хранения параметров, метрик и артефактов запусков. В проекте применялась SQLite-настройка `tracking_uri`, чтобы не зависеть от устаревающего `file-store backend`.

```
mlflow:
  tracking_uri: sqlite:///artifacts/logs/mlflow.db
  experiment_name: room_type_image_heavy
  artifact_root: artifacts/logs/mlruns
```

Тип данных	Что логируется
Params	fold, device, backbone, image_size, batch_size, lr, weight_decay, loss, sampler, augmentation, class_weight_policy, sample_weight_policy
Tags	commit_sha, dataset_version, split_version, seed, feature_flags, tta_flag
Metrics per epoch	train_loss, val_loss, val_macro_f1, val_accuracy
Final metrics	best_val_macro_f1, final_oof_macro_f1, final_shadow_macro_f1, final_shadow_accuracy
Artifacts	checkpoint, fold_oof_predictions.parquet, fold_shadow_predictions.parquet

11. Артефакты обучения и формат OOF

Таблица 11: Артефакты

Артефакт	Назначение
artifacts/checkpoints/*.ckpt	Лучшие веса модели по val_macro_f1
artifacts/oof/<run>/foldN_oof_predictions.parquet	Предсказания fold N на validation
artifacts/oof/<run>/foldN_shadow_predictions.parquet	Предсказания fold N на shadow holdout
artifacts/oof/<run>/oof_predictions.parquet	Объединенные OOF-предсказания
artifacts/oof/<run>/shadow_holdout_predictions.parquet	Агрегированные shadow-предсказания
artifacts/oof/<run>/confusion_matrix_oof.csv	Матрица ошибок
artifacts/oof/<run>/config.yaml	Копия конфига запуска
reports/<run>.md	Отчет по метрикам

Стандартные колонки parquet:

image_id_ext, item_id, result, label, fold, source_dataset, local_path, content_hash,
target, pred,
logit_0 ... logit_19,
prob_0 ... prob_19

12. Inference pipeline

Inference вынесен отдельно от обучения. Релизный конфиг указывает checkpoint, backbone, классы, transforms и device. Основной wrapper - RoomPredictor.

```
Изображение
-> PIL.Image
-> resize / center crop / normalize
-> model.forward()
-> logits
-> softmax probabilities
-> Top-K labels
```

Таблица 12: Инференс

Компонент	Назначение
configs/release/rc1.yaml	Release config с путями к checkpoint и параметрами inference
RoomPredictor.from_config_path	Загрузка модели/моделей из release config
topk_labels(image, k=3)	Возвращает Топ-3 классов с вероятностями
predict.py	CLI-инференс
app.py	Gradio demo

13. Демо: app.py на Gradio

Для демонстрации модели разработан lightweight веб-интерфейс на Gradio. Пользователь загружает изображение комнаты и получает Топ-3 предсказания с вероятностями и техническую информацию о модели.

Таблица 13: Демо

Элемент app.py	Описание
parse_args	Читает --config и --share
DemoPredictor	Обертка над RoomPredictor для Gradio
gr.Image(type="pil")	Входное изображение в формате PIL
gr.Label(num_top_classes=3)	Топ-3 классов
gr.Textbox	config, device, checkpoints
demo.launch(share=args.share)	Локальный или публичный запуск

```
uv run python app.py
uv run python app.py --config configs/release/rc1.yaml
uv run python app.py --share
```

14. Основные команды запуска

Установка зависимостей

```
uv sync --extra dev
```

Проверка данных

```
uv run python src/datasets/make_manifest.py
```

Baseline train

```
uv run python src/training/train_image.py --config configs/model/image_baseline_v1.yaml --all-folds
```

Финальная модель

```
uv run python src/training/train_image.py --config  
configs/model/cv05_rtx5070_xlarge_512_focal_es.yaml --all-folds
```

Debug smoke run

```
uv run python src/training/train_image.py --config  
configs/model/cv05_rtx5070_xlarge_512_focal_es.yaml --fold 0 --debug
```

Demo

```
uv run python app.py --config configs/release/rc1.yaml
```

Линт/тесты

```
uv run ruff check .
```

```
uv run pytest
```


15. Итоговые выводы

В результате работы был сформирован полноценный CV pipeline для классификации изображений помещений. Проект включает подготовку и проверку данных, обучение моделей на PyTorch/timm, поддержку class imbalance techniques, MLflow tracking, OOF/shadow evaluation, release inference и Gradio demo.

Таблица 14: Выводы

Результат	Статус
Проверка датасета и manifest	Реализовано
Train pipeline с fold-ами	Реализовано
Поддержка CE/weighted CE/Focal Loss	Реализовано
Sampler-стратегии для дисбаланса	Реализовано
MLflow SQLite tracking	Реализовано
OOF/shadow parquet export	Реализовано
Финальная heavy-модель	Выбрана: ConvNeXt XLarge 512 focal balanced
Демо-интерфейс	Реализовано через Gradio

Ключевая итоговая конфигурация: ConvNeXt XLarge + вход 512x512 + Focal Loss + balanced sampler + early stopping + AMP + gradient checkpointing. Лучший зафиксированный результат по OOF/macro F1 около 0.7934, что делает эту конфигурацию основной кандидатной моделью для финального решения.